.NET Framework - What's New in C# 7.0

By Mark Michaelis | Connect(); 2016 | Get the Code

Back in December 2015, I discussed the designing of C# 7.0 (msdn.com/magazine/mt595758). A lot has changed over the last year, but the team is now buttoning down C# 7.0 development, and Visual Studio 2017 Release Candidate is implementing virtually all of the new features. (I say virtually because until Visual Studio 2017 actually ships, there's always a chance for further change.) For a brief overview, you can check out the summary table at itl.tc/CSharp7FeatureSummary. In this article I'm going to explore each of the new features in detail.

Deconstructors

Since C# 1.0, it's been possible to call a function—the constructor—that combines parameters and encapsulates them into a class. However, there's never been a convenient way to deconstruct the object back into its constituent parts. For example, imagine a PathInfo class that takes each element of a filename—directory name, file name, extension—and combines them into an object, with support for then manipulating the object's various elements. Now imagine you wish to extract (deconstruct) the object back into its parts.

In C# 7.0 this becomes trivial via the deconstructor, which returns the specifically identified components of the object. Be careful not to confuse a deconstructor with a destructor (deterministic object deallocation and cleanup) or a finalizer (itl.tc/CSharpFinalizers).

Take a look at the PathInfo class in **Figure 1**.

Figure 1 PathInfo Class with a Deconstructor with Associated Tests

```
public class PathInfo
{
  public string DirectoryName { get; }
  public string FileName { get; }
  public string Extension { get; }
  public string Path
  {
    get
    {
      return System.IO.Path.Combine(
        DirectoryName, FileName, Extension);
    }
```

```
  }
  public PathInfo(string path)
  {
    DirectoryName = System.IO.Path.GetDirectoryName(path);
    FileName = System.IO.Path.GetFileNameWithoutExtension(path);
    Extension = System.IO.Path.GetExtension(path);
  }
  public void Deconstruct(
    out string directoryName, out string fileName, out string extension)
  {
    directoryName = DirectoryName;
    fileName = FileName;
    extension = Extension;
  }
  // ...
}
```

Obviously, you can call the Deconstruct method as you would have in C# 1.0. However, C# 7.0 provides syntactic sugar that significantly simplifies the invocation. Given the declaration of a deconstructor, you can invoke it using a new C# 7.0 "tuple-like" syntax (see **Figure 2**).

Figure 2 Deconstructor Invocation and Assignment

```
PathInfo pathInfo = new PathInfo(@"\\test\unc\path\to\something.ext");
{
  // Example 1: Deconstructing declaration and assignment.
  (string directoryName, string fileName, string extension) = pathInfo;
  VerifyExpectedValue(directoryName, fileName, extension);
}
{
  string directoryName, fileName, extension = null;
  // Example 2: Deconstructing assignment.
  (directoryName, fileName, extension) = pathInfo;
  VerifyExpectedValue(directoryName, fileName, extension);
}
{
  // Example 3: Deconstructing declaration and assignment with var.
  var (directoryName, fileName, extension) = pathInfo;
  VerifyExpectedValue(directoryName, fileName, extension);
}
```

Notice how, for the first time, C# is allowing simultaneous assignment to multiple variables of different values. This is not the same as the null assigning declaration in which all variables are initialized to the same value (null):

```
string directoryName, filename, extension = null;
```

Instead, with the new tuple-like syntax, each variable is assigned a different value corresponding not to its name, but to the order in which it appears in the declaration and the deconstruct statement.

As you'd expect, the type of the out parameters must match the type of the variables being assigned, and var is allowed because the type can be inferred from Deconstruct parameter types. Notice, however, that while you can put a single var outside the parentheses as shown in Example 3 in **Figure 2**, at this time it's not possible to pull out a string, even though all the variables are of the same type.
Note that at this time, the C# 7.0 tuple-like syntax requires that at least two variables appear within the parentheses. For example, (FileInfo path) = pathInfo; is not allowed even if a deconstructor exists for:

```
public void Deconstruct(out FileInfo file)
```

In other words, you can't use the C# 7.0 deconstructor syntax for Deconstruct methods with only one out parameter.

Working with Tuples

As I mentioned, each of the preceding examples leveraged the C# 7.0 tuple-like syntax. The syntax is characterized by the parentheses that surround the multiple variables (or properties) that are assigned. I use the term "tuple-like" because, in fact, none of these deconstructor examples actually leverage any tuple type internally. (In fact, assignment of tuples via a deconstructor syntax isn't allowed and arguably would be somewhat unnecessary because the object assigned already is an instance representing the encapsulated constituent parts.)

With C# 7.0 there's now a special streamlined syntax for working with tuples, as shown in **Figure 3**. This syntax can be used whenever a type specifier is allowed, including declarations, cast operators and type parameters.

Figure 3 Declaring, Instantiating and Using the C# 7.0 Tuple Syntax

```
[TestMethod]
public void Constructor_CreateTuple()
{
  (string DirectoryName, string FileName, string Extension) pathData =
    (DirectoryName: @"\\test\unc\path\to",
    FileName: "something",
    Extension: ".ext");
  Assert.AreEqual<string>(
    @"\\test\unc\path\to", pathData.DirectoryName);
  Assert.AreEqual<string>(
    "something", pathData.FileName);
```

```
  Assert.AreEqual<string>(
    ".ext", pathData.Extension);
  Assert.AreEqual<(string DirectoryName, string FileName, string
Extension)>(
    (DirectoryName: @"\\test\unc\path\to",
      FileName: "something", Extension: ".ext"),
    (pathData));
  Assert.AreEqual<(string DirectoryName, string FileName, string
Extension)>(
    (@"\\test\unc\path\to", "something", ".ext"),
    (pathData));
  Assert.AreEqual<(string, string, string)>(
    (@"\\test\unc\path\to", "something", ".ext"), (pathData));
  Assert.AreEqual<Type>(
    typeof(ValueTuple<string, string, string>), pathData.GetType());
}
[TestMethod]
public void ValueTuple_GivenNamedTuple_ItemXHasSameValuesAsNames()
{
  var normalizedPath =
    (DirectoryName: @"\\test\unc\path\to", FileName: "something",
    Extension: ".ext");
  Assert.AreEqual<string>(normalizedPath.Item1,
normalizedPath.DirectoryName);
  Assert.AreEqual<string>(normalizedPath.Item2,
normalizedPath.FileName);
  Assert.AreEqual<string>(normalizedPath.Item3,
normalizedPath.Extension);
}
static public (string DirectoryName, string FileName, string Extension)
  SplitPath(string path)
{
  // See http://bit.ly/2dmJIMm Normalize method for full implementation.
  return (
    System.IO.Path.GetDirectoryName(path),
    System.IO.Path.GetFileNameWithoutExtension(path),
    System.IO.Path.GetExtension(path)
    );
}
```

For those not familiar with tuples, it's a way of combining multiple types into a single containing type in a lightweight syntax that's available outside of the method in which it's instantiated. It's lightweight because, unlike defining a class/struct, tuples can be "declared" inline and on the fly. But, unlike dynamic types that also support inline declaration and instantiation, tuples can be accessed outside of their containing member and, in fact, they can be included as part of an API. In spite of the external

API support, tuples don't have any means of version-compatible extension (unless the type parameters themselves happen to support derivation), thus, they should be used with caution in public APIs. Therefore, a preferable approach might be to use a standard class for the return on a public API.

Prior to C# 7.0, the framework already had a tuple class, System.Tuple<…> (introduced in the Microsoft .NET Framework 4). C# 7.0 differs from the earlier solution, however, because it embeds the semantic intent into declaration and it introduces a tuple value type: System.ValueTuple<…>.

Let's take a look at the semantic intent. Notice in **Figure 3** that the C# 7.0 tuple syntax allows you to declare alias names for each ItemX element the tuple contains. The pathData tuple instance in **Figure 3**, for example, has strongly typed DirectoryName: string, FileName: string, and Extension: string properties defined, thus allowing calls to pathData.DirectoryName, for example. This is a significant enhancement because prior to C# 7.0, the only names available were ItemX names, where the X incremented for each element.
Now, while the elements for a C# 7.0 tuple are strongly typed, the names themselves aren't distinguishing in the type definition. Therefore, you can assign two tuples with disparate name aliases and all you'll get is a warning that informs you the name on the right-hand side will be ignored:

```
// Warning: The tuple element name 'AltDirectoryName1' is ignored
// because a different name is specified by the target type...
(string DirectoryName, string FileName, string Extension) pathData =
  (AltDirectoryName1: @"\\test\unc\path\to",
  FileName: "something", Extension: ".ext");
```

Similarly, you can assign tuples to other tuples that may not have all alias element names defined:

```
// Warning: The tuple element name 'directoryName', 'FileNAme' and 'Extension'
// are ignored because a different name is specified by the target type...
(string, string, string) pathData =
  (DirectoryName: @"\\test\unc\path\to", FileName: "something", Extension:
".ext");
```

To be clear, the type and order of each element does define type compatibility. Only the element names are ignored. However, even though ignored when they're different, they still provide IntelliSense within the IDE.

Note that, whether or not an element name alias is defined, all tuples have ItemX names where X corresponds to the number of the element. The ItemX names are

important because they make the tuples available from C# 6.0, even though the alias element names are not.

Another important point to be aware of is that the underlying C# 7.0 tuple type is a System.ValueTuple. If no such type is available in the framework version you're compiling against, you can access it via a NuGet package.

For details about the internals of tuples, check see intellitect.com/csharp7tupleiinternals.

Pattern Matching with Is Expressions

On occasion you have a base class, Storage for example, and a series of derived classes, DVD, UsbKey, HardDrive, FloppyDrive (remember those?) and so on. To implement an Eject method for each you have several options:

- As Operator
  - Cast and assign using the as operator
  - Check the result for null
  - Perform the eject operation
- Is Operator
  - Check the type using the is operator
  - Cast and assign the type
  - Perform the eject operation
- Cast
  - Explicit cast and assign
  - Catch possible exception
  - Perform operation
  - Yuck!

There's a fourth, far-better approach using polymorphism in which you dispatch using virtual functions. However, this is available only if you have the source code for the Storage class and can add the Eject method. That's an option I'm assuming is unavailable for this discussion, hence the need for pattern matching.

The problem with each of these approaches is that the syntax is fairly verbose and always requires multiple statements for each class to which you want to cast. C# 7.0 provides pattern matching as a means of combining the test and the assignment into a single operation. As a result, the code in **Figure 4** simplifies down to what's shown in **Figure 5**.

Figure 4 Type Casting Without Pattern Matching

```
// Eject without pattern matching.
public void Eject(Storage storage)
{
  if (storage == null)
  {
    throw new ArgumentNullException();
  }
  if (storage is UsbKey)
  {
    UsbKey usbKey = (UsbKey)storage;
    if (usbKey.IsPluggedIn)
    {
      usbKey.Unload();
      Console.WriteLine("USB Drive Unloaded.");
    }
    else throw new NotImplementedException();     }
  else if(storage is DVD)
  // ...
  else throw new NotImplementedException();
}
```

Figure 5 Type Casting with Pattern Matching

```
// Eject with pattern matching.
public void Eject(Storage storage)
{
  if (storage is null)
  {
    throw new ArgumentNullException();
  }
  if ((storage is UsbKey usbDrive) && usbDrive.IsPluggedIn)
  {
    usbDrive.Unload();
    Console.WriteLine("USB Drive Unloaded.");
  }
  else if (storage is DVD dvd && dvd.IsInserted)
  // ...
  else throw new NotImplementedException();   // Default
}
```

The difference between the two isn't anything radical, but when performed frequently (for each of the derived types, for example) the former syntax is a burdensome C# idiosyncrasy. The C# 7.0 improvement—combining the type test, declaration and assignment into a single operation—renders the earlier syntax all but deprecated. In the former syntax, checking the type without assigning an identifier makes falling through to the "default" else cumbersome at best. In contrast, the assignment allows for the additional conditionals beyond just the type check.

Note that the code in **Figure 5** starts out with a pattern-matching is operator with support for a null comparison operator, as well:

```
if (storage is null) { ... }
```

Pattern Matching with the Switch Statement

While supporting pattern matching with the is operator provides an improvement, pattern-matching support for a switch statement is arguably even more significant, at least when there are multiple compatible types to which to convert. This is because C# 7.0 includes case statements with pattern matching and, furthermore, if the type pattern is satisfied in the case statement, an identifier can be provided, assigned, and accessed all within the case statement. **Figure 6** provides an example.

Figure 6 Pattern Matching in a Switch Statement

```
public void Eject(Storage storage)
{
  switch(storage)
  {
    case UsbKey usbKey when usbKey.IsPluggedIn:
      usbKey.Unload();
      Console.WriteLine("USB Drive Unloaded.");
      break;
    case DVD dvd when dvd.IsInserted:
      dvd.Eject();
      break;
    case HardDrive hardDrive:
      throw new InvalidOperationException();
    case null:
    default:
      throw new ArgumentNullException();
  }
}
```

Notice in the example how local variables like usbKey and dvd are declared and assigned automatically within the case statement. And, as you'd expect, the scope is limited to within the case statement.

Perhaps just as important as the variable declaration and assignment, however, is the additional conditional that can be appended to the case statement with a when clause. The result is that a case statement can completely filter out an invalid scenario without an additional filter inside the case statement. This has the added advantage of allowing evaluation of the next case statement if, in fact, the former case statement is not fully met. It also means that case statements are no longer limited to constants and,

furthermore, a switch expression can be any type—it's no longer limited to bool, char, string, integral and enum.

Another important characteristic the new C# 7.0 pattern-matching switch statement capability introduces is that case statement order is significant and validated at compile time. (This is in contrast with earlier versions of the language, in which, without pattern matching, case statement order was not significant.) For example, if I introduced a case statement for Storage prior to a pattern-matching case statement that derives from Storage (UsbKey, DVD and HardDrive), then the case Storage would eclipse all other type pattern matching (that derives from Storage). A case statement from a base type that eclipses other derived type case statements from evaluation will result in a compile error on the eclipsed case statement. In this way, case statement order requirements are similar to catch statements.

Readers will recall that an is operator on a null value will return false. Therefore, no type pattern-matching case statement will match for a null-valued switch expression. For this reason, order of the null case statement won't matter; this behavior matches switch statements prior to pattern matching.

Also, in support of compatibility with switch statements prior to C# 7.0, the default case is always evaluated last regardless of where it appears in the case statement order. (That said, readability would generally favor putting it at the end, because it's always evaluated last.) Also, goto case statements still work only for constant case labels—not for pattern matching.

Local Functions

While it's already possible to declare a delegate and assign it an expression, C# 7.0 takes this one step further by allowing the full declaration of a local function inline within another member. Consider the IsPalindrome function in **Figure 7**.

Figure 7 A Local Function Example

```
bool IsPalindrome(string text)
{
  if (string.IsNullOrWhiteSpace(text)) return false;
  bool LocalIsPalindrome(string target)
  {
    target = target.Trim();   // Start by removing any surrounding whitespace.
    if (target.Length <= 1) return true;
    else
    {
      return char.ToLower(target[0]) ==
        char.ToLower(target[target.Length - 1]) &&
```

```
        LocalIsPalindrome(
          target.Substring(1, target.Length - 2));
    }
  }
  return LocalIsPalindrome(text);
}
```

In this implementation, I first check that the argument passed to IsPalindrome isn't null or only whitespace. (I could've used pattern matching with "text is null" for the null check.) Next, I declare a function LocalIsPalindrome in which I compare the first and last characters recursively. The advantage of this approach is that I don't declare the LocalIsPalindrome within the scope of the class where it can potentially be called mistakenly, thus circumventing the IsNullOrWhiteSpace check. In other words, local functions provide an additional scope restriction, but only inside the surrounding function.

The parameter validation scenario in **Figure 7** is one of the common local function use cases. Another one I encounter frequently occurs within unit tests, such as when testing the IsPalindrome function (see **Figure 8**).

Figure 8 Unit Testing Often Uses Local Functions

```
[TestMethod]
public void IsPalindrome_GivenPalindrome_ReturnsTrue()
{
  void AssertIsPalindrome(string text)
  {
    Assert.IsTrue(IsPalindrome(text),
      $"'{text}' was not a Palindrome.");
  }
  AssertIsPalindrome("7");
  AssertIsPalindrome("4X4");
  AssertIsPalindrome("  tnt");
  AssertIsPalindrome("Was it a car or a cat I saw");
  AssertIsPalindrome("Never odd or even");
}
```

Iterator functions that return IEnumerable<T> and yield return elements are another common local function use case.

To wrap up the topic, here are a few more points to be aware of for local functions:

- Local functions don't allow use of an accessibility modifier (public, private, protected).
- Local functions don't support overloading. You can't have two local functions in the same method with the same name even if the signatures don't overlap.

- The compiler will issue a warning for local functions that are never invoked.
- Local functions can access all variables in the enclosing scope, including local variables. This behavior is the same with locally defined lambda expressions except that local functions don't allocate an object that represents the closure, as locally defined lambda expressions do.
- Local functions are in scope for the entire method, regardless of whether they're invoked before or after their declaration.

Return by Reference

Since C# 1.0 it has been possible to pass arguments into a function by reference (ref). The result is that any change to the parameter itself will get passed back to the caller. Consider the following Swap function:

```
static void Swap(ref string x, ref string y)
```

In this scenario, the called method can update the original caller's variables with new values, thereby swapping what's stored in the first and second arguments.

Starting in C# 7.0, you're also able to pass back a reference via the function return—not just a ref parameter. Consider, for example, a function that returns the first pixel in an image that's associated with red-eye, as shown in **Figure 9**.

Figure 9 Ref Return and Ref Local Declaration

```
public ref byte FindFirstRedEyePixel(byte[] image)
{
  //// Do fancy image detection perhaps with machine learning.
  for (int counter = 0; counter < image.Length; counter++)
  {
    if(image[counter] == (byte)ConsoleColor.Red)
    {
      return ref image[counter];
    }
  }
  throw new InvalidOperationException("No pixels are red.");
}
[TestMethod]
public void FindFirstRedEyePixel_GivenRedPixels_ReturnFirst()
{
  byte[] image;
  // Load image.
  // ...
    // Obtain a reference to the first red pixel.
  ref byte redPixel = ref FindFirstRedEyePixel(image);
```

```
  // Update it to be Black.
  redPixel = (byte)ConsoleColor.Black;
  Assert.AreEqual<byte>((byte)ConsoleColor.Black, image[redItems[0]]);
}
```

By returning a reference to the image, the caller is then able to update the pixel to a different color. Checking for the update via the array shows that the value is now black. The alternative of using a by reference parameter is, one might argue, less obvious and less readable:

```
public bool FindFirstRedEyePixel(ref byte pixel);
```

There are two important restrictions on return by reference—both due to object lifetime. These are that object references shouldn't be garbage collected while they're still referenced, and they shouldn't consume memory when they no longer have any references. First, you can only return references to fields, other reference-returning properties or functions, or objects that were passed in as parameters to the by reference-returning function. For example, FindFirstRedEyePixel returns a reference to an item in the image array, which was a parameter to the function. Similarly, if the image was stored as a field within a class, you could return the field by reference:

```
byte[] _Image;
public ref byte[] Image { get {  return ref _Image; } }
```

Second, ref locals are initialized to a certain storage location in memory, and can't be modified to point to a different location. (You can't have a pointer to a reference and modify the reference—a pointer to a pointer for those of you with a C++ background.)

There are several return-by-reference characteristics of which to be cognizant:

- If you're returning a reference you obviously have to return it. This means, therefore, that in the example in **Figure 9**, even if no red-eye pixel exists, you still need to return a ref byte. The only workaround would be to throw an exception. In contrast, the by reference parameter approach allows you to leave the parameter unchanged and return a bool indicating success. In many cases, this might be preferable.
- When declaring a reference local variable, initialization is required. This involves assigning it a ref return from a function or a reference to a variable:

```
ref string text;   // Error
```

- Although it's possible in C# 7.0 to declare a reference local variable, declaring a field of type ref isn't allowed:

```
class Thing { ref string _Text;   /* Error */ }
```

- You can't declare a by reference type for an auto-implemented property:

```
class Thing { ref string Text { get;set; }   /* Error */ }
```

- Properties that return a reference are allowed:

```
class Thing { string _Text = "Inigo Montoya";
  ref string Text { get { return ref _Text; } } }
```

- A reference local variable can't be initialized with a value (such as null or a constant). It must be assigned from a by reference returning member or a local variable/field:

```
ref int number = null; ref int number = 42;   // ERROR
```

Out Variables

Since the first release of C#, the invocation of methods containing out parameters always required the pre-declaration of the out argument identifier before invocation of the method. C# 7.0 removes this idiosyncrasy, however, and allows the declaration of the out argument inline with the method invocation. **Figure 10** shows an example.

Figure 10 Inline Declaration of Out Arguments

```
public long DivideWithRemainder(
  long numerator, long denominator, out long remainder)
{
  remainder = numerator % denominator;
  return (numerator / denominator);
}
[TestMethod]
public void DivideTest()
{
  Assert.AreEqual<long>(21,
    DivideWithRemainder(42, 2, out long remainder));
  Assert.AreEqual<long>(0, remainder);
}
```

Notice how in the DivideTest method, the call to DivideWithRemainder from within the test includes a type specifier after the out modifier. Furthermore, see how remainder continues to be in scope of the method automatically, as evidenced by the second Assert.AreEqual invocation. Nice!

Literal improvements

Unlike previous versions, C# 7.0 includes a numeric binary literal format, as the following example demonstrates:

```
long LargestSquareNumberUsingAllDigits =
```

```
  0b0010_0100_1000_1111_0110_1101_1100_0010_0100;   // 9,814,072,356
long MaxInt64 { get; } =
  9_223_372_036_854_775_807;   // Equivalent to long.MaxValue
```

Notice also the support for the underscore "_" as a digit separator. It's used simply to improve readability and can be placed anywhere between the digits of the number—binary, decimal or hexadecimal.

Generalized Async Return Types

On occasion when implementing an async method, you're able to return the result synchronously, short-circuiting a long-running operation because the result is virtually instantaneous or even already known. Consider, for example, an async method that determines the total size of files within a directory (bit.ly/2dExeDG). If, in fact, there are no files in the directory, the method can return immediately without ever executing a long-running operation. Until C# 7.0, the requirements of async syntax dictated that the return from such a method should be a Task<long> and, therefore, that a Task be instantiated even if no such Task instance is required. (To achieve this, the general pattern is to return the result from Task.FromResult<T>.)
In C# 7.0, the compiler no longer limits async method returns to void, Task or Task<T>. You can now define custom types, such as the .NET Core Framework-provided System.Threading.Tasks.ValueTask<T> struct, which are compatible with an async method return. See itl.tc/GeneralizedAsyncReturnTypes for more information.

More Expression-Bodied Members

C# 6.0 introduced expression-bodied members for functions and properties, enabling a streamlined syntax for implementing trivial methods and properties. In C# 7.0, expression-bodied implementations are added to constructors, accessors (get and set property implementations) and even finalizers (see **Figure 11**).

Figure 11 Using Expression-Bodied Members in Accessors and Constructors

```
class TemporaryFile   // Full IDisposible implementation
                      // left off for elucidation.
{
  public TemporaryFile(string fileName) =>
    File = new FileInfo(fileName);
  ~TemporaryFile() => Dispose();
  Fileinfo _File;
  public FileInfo File
  {
    get => _File;
```

```
    private set => _File = value;
  }
  void Dispose() => File?.Delete();
}
```

I expect the use of expression-bodied members to be particularly common for finalizers because the most common implementation is to call the Dispose method, as shown.

I'm pleased to point out that the additional support for expression-bodied members was implemented by the C# community rather than the Microsoft C# team. Yay for open source!

Caution: This feature is not implemented in Visual Studio 2017 RC.

Throw Expressions

The Temporary class in **Figure 11** can be enhanced to include parameter validation within the expression-bodied members; therefore, I can update the constructor to be:

```
public TemporaryFile(string fileName) =>
  File = new FileInfo(filename ?? throw new ArgumentNullException());
```

Without throw expressions, C# support for expression-bodied members couldn't do any parameter validation. However, with C# 7.0 support for throw as an expression, not just a statement, the reporting of errors inline within larger containing expressions becomes possible.

Caution: This feature is not implemented in Visual Studio 2017 RC.

Wrapping Up

I confess that when I started writing this article, I expected it to be much shorter. However, as I spent more time programming and testing the features, I discovered there was way more to C# 7.0 than I realized from reading the feature titles and following the language development. In many cases—declaring out variables, binary literals, throw expressions and such—there isn't much involved in understanding and using the features. However, several cases—return by reference, deconstructors and tuples, for example—require much more to learn the feature than one might expect initially. In these latter cases, it isn't just the syntax, but also knowing when the feature is relevant.

C# 7.0 continues to whittle away at the quickly decreasing list of idiosyncrasies (pre-declared out identifiers and lack of throw expressions), while at the same time

broadening to include support for features previously not seen at the language level (tuples and pattern matching).

Hopefully, this introduction helps you jump into programming C# 7.0 immediately. For more information on C# 7.0 developments following this writing, check out my blog at intellitect.com/csharp7, as well as an update to my book, "Essential C# 7.0" (which is expected to come out shortly after Visual Studio 2017 is released to manufacturing).

Mark Michaelis *is founder of IntelliTect, where he serves as its chief technical architect and trainer. For nearly two decades he has been a Microsoft MVP and a Microsoft Regional Director since 2007. Michaelis serves on several Microsoft software design review teams, including C#, Microsoft Azure, SharePoint and Visual Studio ALM. He speaks at developer conferences and has written numerous books including his most recent, "Essential C# 6.0 (5th Edition)" (itl.tc/EssentialCSharp). Contact him on Facebook at facebook.com/Mark.Michaelis, on his blog at IntelliTect.com/Mark, on Twitter: @markmichaelis or via e-mail at mark@IntelliTect.com.*
Thanks to the following technical experts for reviewing this article: Kevin Bost (IntelliTect), Mads Torgersen (Microsoft) and Bill Wagner (Microsoft)

Discuss this article in the MSDN Magazine forum

Go to post

The ref variable can be modified point to other location or not? Following paragraph reference from '.NET Framework - What's New in C# 7.0' : Second, ref locals are initialized to a certain storage location in memory, and can't be modified to poi...

Mar 10, 2017

Go to post

Thanks for taking the time to comment Jeff. This is great and I think your comments make some valid points that I will try to respond to in order. But one egregious example is in the new pattern matching section. The example argues for the use of t...

Dec 14, 2016

Go to post

When I first read of these changes, my concern was that it felt like C# was turning into JavaScript and this update does little to convince me otherwise. Most of these enhancements seem to do little to improve the readability and maintainability of t...

Dec 12, 2016

Go to post

This link mentioned at the beginning of the article is currently dead: http://itl.tc/CSharp7FeatureSummary

Dec 7, 2016

Go to post

Even more ((()(())())) brackets with tuples. They make C# less readable. Like C/C++ typecasts in former days or LISP.

Dec 2, 2016

Go to post

By the way, do note that there is an extra parenthesis in the if statement that should not be there. if (((storage is UsbKey usbDrive) && usbDrive.IsPluggedIn) ---^

Nov 30, 2016

Go to post

Hi DarKlajid, Yup.. .that is correct. Thanks for pointing it out. I will try to get it updated. Mark

Nov 17, 2016

Go to post

Figure 5 / Type casting with Pattern Matching contains this: if (((storage is UsbKey usbDrive) && usbDrive.IsPluggedIn) { usbKey.Unload();I assume the last line should read usbDrive.Unload() instead.

Nov 16, 2016

Go to post

The team is now buttoning down development on C# 7.0, and Visual Studio 2017 Release Candidate is implementing virtually all of the new features. Come along as Mark Michaelis explores each of the new features in detail, including deconstructors, patt...

Nov 16, 2016